

日程与日记助手 (Diary Assistant) PRD

用户痛点

现代人在日常工作和生活中经常面临日程繁杂、难以追踪个人时间分配的问题。传统的日程管理软件只能起到提醒的作用，而用户常常忘记记录自己到底做了什么、做事的感受如何。同时，手动写日记门槛较高，难以坚持。用户迫切需要一个能结合其真实执行情况（如电脑应用使用状况）与日程安排的智能助手，帮助自己进行反思、回顾，并自动沉淀为个人的日记。

解决方案

设计一个“日程管理 + AI日记助手”的桌面级应用程序。该应用不仅允许用户管理个人的日程（普通日程和固定日程），还能通过轻量级的应用数据采集机制（基于严格白名单的应用使用情况追踪），自动捕获用户在关键任务上的实际聚焦时间。在一天结束时，系统会将当天的日程安排、应用使用记录以及用户的心情/感言结合起来，利用大语言模型（LLM）的推理能力自动生成总结性日记，大大降低了日记记录的门槛。

功能拆解

1. 日程管理

功能名称	功能描述	重要性
添加日程	用户可以在当前日期下添加多个日程，每个日程包含日程类别、开始时间、持续时间、日程描述，以及是否为“固定日程”	必要
编辑日程	用户可以对未完成日程进行编辑，可编辑内容包括日程类别、持续时间和日程描述，以及是否为“固定日程”	必要
删除日程	用户可以删除日程，系统在用户删除日程时应当弹窗进行二次确认，确认后删除日程	必要
完成感言	用户可以在日程结束后对该日程进行评价或描述	重要

功能名称	功能描述	重要性
日程分类	日程包含普通日程与固定日程,普通日程仅在日程被创建的目标日期生效，固定日程可以设定频次，以固定频次生效（如每天，每周二、四，每个月1号、16号，每年等）	重要
日程闹钟	用户可以给日程设置闹钟（默认不设置），用户可指定闹钟的响铃时机（如开始时间，或开始时间前5分钟/半小时/一小时）会在日程所在日期	次要

2. 日记功能

功能名称	功能描述	重要性
信息汇总	用户可以在日记草稿页面查看自己当日的信息汇总，该汇总信息结合了当日日程的完成情况、用户的当日应用使用情况和用户对日程的完成感言	必要
草稿修改	用户在查看信息汇总后，可以根据自己的喜好对里面的内容进行编辑，例如删去部分应用的使用记录，或者新增一些感想，并对修改的内容进行确认和保存	必要
草稿上传	用户在确认草稿信息后，可以手动选择上传草稿，应用会将草稿和一些对日记风格等额外需求的内容打包成prompt，提交给大模型进行文本生成。	必要
日记自定义	用户可以选择对日记的文风、格式进行自定义，并将其应用于当前或后续所有的日记中。应用会提供制式的填充式问卷来让用户进行自定义，并将自定义结果作为prompt的一部分在生成日记时上传给大模型	重要
自动上传	用户可以选择是否开启自动上传功能。用户可以设定每日的自动上传时间，或者在第二天自动总结并生成日记。	次要

3. 健康管理（次要）

功能名称	功能描述	重要性
久坐计时	用户可以选择是否开启久坐计时功能。久坐计时包含“久坐倒计时”和“起身运动”两个部分的计时，可由用户分别设定并在开启后循环切换。“久坐倒计时”倒计时结束后会以消息的方式提醒用户起身运动，“起身运动”倒计时结束后会以消息的方式提醒用户可以坐下继续工作	次要
用眼疲劳	用户可以选择是否开启用眼疲劳提醒功能。用眼疲劳提醒倒计时结束后会以消息的方式提醒用户需要眺望远方放松20秒，30秒后自动重新开始倒计时	次要

4. 前端组件

页面名称	页面描述	交互路径
桌面组件	显示当日日程和健康管理倒计时	显示在桌面，点击可进入后台组件详细编辑
后台组件-日程编辑	可以实现日程管理功能，同时支持今日日程和全年日历视图切换	点击桌面组件进入，打开软件后默认显示日程编辑页面，可从其他页面点击左侧导航栏“日程”图标进入
后台组件-健康管理	可以实现健康管理功能	点击对应健康管理倒计时桌面组件进入，可从其他页面点击左侧导航栏“健康管理”图标进入
后台组件-日记功能	可以实现当日日记信息汇总和日记编辑功能	点击页面左侧导航栏“日记”图标进入
后台组件-历史日记	视图为日历形式，点击日期可以查看当日的日程和日历，可以手动上传补充日记	可从其他页面点击左侧导航栏“日历”图标进入
后台组件-个人主页	可以查看一些基础设置（如APIkey管理，日记文风格设置）	可从其他页面点击左侧导航栏“个人”图标进入

用户故事

- 1. 作为一名用户，我希望能够添加带有类别、时间、描述的日程，以便于规划我的一天。
- 2. 作为一名用户，我希望能够设置固定日程模板，这样我不需要每天重复添加相同的日常任务（如每日晨会）。
- 3. 作为一名用户，我希望能够修改某一天固定日程的具体实例（比如改时间），而不影响未来或过去的日程模板，以便灵活应对突发情况。

- 4. 作为一名注重隐私的用户，我希望能够在总的白名单界面中选择授权记录哪些应用，只让系统记录我想追踪的工作软件。
- 5. 作为一名注重效率的用户，我希望系统只在我活跃坐在电脑前时才进行健康计时（久坐、用眼疲劳），而在我离开或电脑休眠时自动挂起，以免发出无效的压迫式提醒。
- 6. 作为一名用户，我希望健康提醒只起到提示作用，并提供“提前进入下一阶段”或“重新开始计时”按钮，这样我能灵活掌控自己的节奏。
- 7. 作为一名用户，我希望可以在日记的“草稿阶段”浏览今天所有将发送给 AI 的数据（日程、应用记录、感言），以便我在发送前可以删减隐私信息。
- 8. 作为一名用户，我希望可以一键将草稿发送给大模型生成日记，并在生成后预览初稿，如果文风不满意能够点击“重新生成”。
- 9. 作为一名用户，我希望能够对大模型生成的日记进行手动微调，并在微调后将其保存为不可轻易丢失的“定稿”。
- 10. 作为一名追求自动化的用户，我希望可以开启“自动上传”功能，到了设定时间系统会自动将当日草稿打包发给大模型并直接存储为定稿日记，免去我每天手动点击的繁琐。
- 11. 作为一名用户，我希望当系统执行后台“自动定稿”时，我不被任何弹窗打扰；但当我主动在设置中开启该功能，或者手动点击上传时，系统能弹窗向我明确提示隐私风险并进行二次确认。
- 12. 作为一名用户，我希望如果我没有开启自动上传功能，系统在一天结束时能将未处理的“草稿”直接降级作为当日的正式日记数据登记保存，方便我以后回顾和二次编辑。
- 13. 作为一名用户，我希望即使是已经定稿的历史日记，我依然可以通过历史日历视图去查看和再次编辑它们。

实现方案

功能实现

- **数据解耦与提示词工程**：日程记录与应用使用情况记录在底层数据结构中保持松耦合。两者不进行硬性的结构绑定，而是在上传阶段，将两者作为独立上下文拼接进大模型 Prompt，由大模型自行进行推理和信息融合。
- **模板与实例分离架构 (Template & Instance)**：“固定日程”作为规则模板配置储存于后台，而在日历中展示和编辑的均为基于模板生成的“实例”。对实例的单个修改脱离模板；但在实例的编辑界面应提供快捷入口直接跳转至模板修改。
- **日记生命周期状态机**：日记的生命周期被严格分为三个状态：**Draft** (草稿，含生数据，可删改), **Generated** (AI生成初稿，提供“重新生成”), **Finalized** (定稿保存，作为永久记录)。
- **无阻断自动化策略**：“自动上传”逻辑分离了授权和执行。开启功能时进行一次性弹窗二次确认风险提示。每日自动执行时不弹窗（静默执行）。触发自动上传时：若当日日记为 **Draft**，则直接执行生成并转至 **Finalized**；若已经处于 **Generated**，则挂起等待人工确认。
- **降级登记机制**：未开启自动上传的情况下，结束当天后，**Draft** 状态的数据直接作为当天的正式日记进行转储登记，日后依然可二次编辑。
- **隐私白名单 (Opt-in)**：应用使用情况数据的收集，强制采用应用白名单模式。首次使用时必须引导用户设置该名单。
- **基于挂起的死计时**：通过系统钩子（检测睡眠唤醒状态等）判定活动。健康计时采用“Wall-clock死计时”，但在系统休眠时挂起，唤醒后继续；附带手动控制按钮以减轻监控感。

程序架构

```
Diary_Assistant
├── Backend/      # 后端
```

— Data/	# 数据储存 (如个人日志、图标等)
— Frontend/	# 前端页面 (后台组件部分)
— Desktop/	# 桌面组件
— Document/	# 项目文档
— README.md	# 项目说明文档

测试方案

- 重点测试日记生命周期的状态机切换 (Draft -> Generated -> Finalized) , 尤其是手动流转与自动流转的边界条件测试 (例如: 到达自动上传时间时, 日记处于不同状态下的表现与挂起逻辑) 。
- 测试“固定日程模板”的生成逻辑, 以及修改当天实例时, 是否正确地仅对实例生效, 而不污染模板全局。
- 模拟系统休眠和唤醒事件, 验证“健康管理倒计时”的自动挂起和恢复逻辑是否精准。
- 测试系统的弹窗拦截与授权状态: 确认手动上传及初次开启“自动上传”时的弹窗提示, 以及开启后触发静默自动上传的不阻断表现。
- 测试应用底层捕获逻辑时, 以外部行为验证为主, 验证上传的 Payload 是否仅仅集成了白名单内的应用和相关日程。

功能忽略

- 本版本不包括多设备同步与云端跨端功能, 数据和日记内容暂以本地储存为主。
- 复杂的日记多媒体嵌入 (如直接插入视频、大量图片图文混排) 不在 MVP 范围内。
- 全自动的本地 NLP 隐私脱敏审查服务不在范围内 (依赖于首次开启确认及白名单机制的“责任自担”) 。

未来路线

- 鉴于应用需要长期在后台合法获取操作系统的活跃的前台窗口名称以及系统睡眠/唤醒钩子, 建议后端采取对原生 API 支持较好的技术栈, 如 `Node.js` / `Electron` 的原生绑定, 或者采用单独挂载在后台的 `Python` 服务 (例如利用 `pywin32` 在 Windows 上抓取) 与 UI 层通信。